Assessment 3 Assignment – Reflection

Yogita Kumari Roll No. 230101091 CS4032 Cloud Computing

GitHub repository: `https://github.com/yogitaranwa/CS4032A3`

When I started this assignment, I had only a theoretical idea of how machine learning models are actually deployed. Building a natural-language command parser for Kubernetes end to end taught me how many different pieces have to fit together before a model becomes a real service: data labelling, training, packaging, CI/CD, container orchestration and load testing. The measurements I quote below are the ones stored in the `results/` folder. They come from the 30 October run of the same code in a separate rehearsal repository, and I say so because a number is only as honest as its origin.

The first thing I learnt was that data quality decides everything. We needed 3,000 labelled commands, and labelling them by hand would have been impossible. I used a teacher model (phi3.5) to label them, and split the work into 15 shards of 200 commands each so the labelling could run in parallel and be re-run safely if a shard failed. Because each shard is cleaned and merged by a separate step, a step can be retried without corrupting the output, and that idempotent, resumable design made failures far less frightening.

The cleaning stage taught me an even harder lesson. Of the 3,000 labelled records, 152 were rejected because the slot text the teacher returned was not a phrase of the command, 44 were duplicates, and 2,804 were kept. None was rejected for invalid JSON. When I read the rejections, the pattern was clear: the teacher sometimes paraphrased a slot instead of copying the words. The right response was not to edit the teacher's answers, which would be dishonest, but to drop those records and, as an improvement, to ask the teacher to repair only the rejected ones or to constrain its output to a JSON grammar. I learnt to treat the dataset as something engineered, not something given, and to measure how much data actually survives cleaning instead of assuming the input count is the usable count.

The second big lesson was that a model is only as trustworthy as the gate that lets it through. The pipeline has linting, secret scanning, unit tests, a network ACL check, model evaluation and a quality gate that requires at least 95% valid JSON. The stored scores show why such a gate matters: the untrained base model produced 0% valid

JSON, while both fine-tuned students reached 99%. The base model would therefore be rejected, although I did not store a CI run of it. Seeing that difference convinced me that evaluation should be automated and enforced, not done by eye.

Comparing models on the same 100 validation examples was also satisfying. The untrained base model scored zero on every metric, while my fine-tuned versions produced valid structured output 99 times out of 100, which is the clearest proof I have produced that fine-tuning actually did something. On the strict full-frame metric the two students scored 50 and 52, and the teacher scored 48. That looks like the students beating their teacher, but with only 100 examples the differences are within noise, so I do not claim more than that the students match the teacher they were trained on.

Packaging the service was a lesson in constraints. The Docker image had to stay under 2 GB while carrying a language model, a tokenizer and a web server. It came to 1.22 GB, using a pinned slim Python base image and removing the pip cache after installing the requirements. Most of that size is probably the PyTorch runtime, which I assumed and did not measure. I also learnt why model artefacts belong in a release with checksums rather than inside git: the repository stays light, every model is verifiable by its hash, and the service checks those hashes before it loads the model.

Deployment brought its own checks. The service had to answer 50 requests within 1 CPU and 1 GiB of memory. In the stored run all 50 requests returned HTTP 200, the container was not killed for memory, and it peaked at 358.7 MiB, which left comfortable headroom. Reading the request log taught me something subtler: request 4, "what time does meeting start", returned valid JSON with no slots, which is a wrong answer that still passes the schema check. A validity gate checks the shape of a reply and not its truth, so correctness has to be measured separately by the scoring script. The log records the index, status code and body of every request, which makes any failure traceable from the log alone.

The Kubernetes scenarios taught me how systems behave under real pressure. In the autoscaling run, 615 requests completed with no failures, and the horizontal pod autoscaler scaled the service from 1 to 2 replicas and back to 1. In the standard run the two replicas received almost the same number of requests (248 and 250), which made load balancing feel real for the first time. In the rolling update to the second model version, 1 of 906 requests failed (0.110%), although the shipped manifest sets `maxUnavailable` to 0, a 10 s `preStop` delay and a 30 s grace period (the stored results

do not show whether the 30 October run used these settings). Those settings reduce errors, but one request is still too few to measure a true failure rate. In the recovery scenario a deleted Pod was replaced in 7 s with no failed request out of 708.

Measuring cost was a smaller task but changed how I think. Working out the price per thousand requests from the rate card and the measured throughput gave about 0.53 INR. That figure rests on a hypothetical rate card, so it is an assumption, whereas the throughput of 4.16 requests per second is a measurement. I now understand why engineering reports separate measured results from assumptions: a number you did not measure is a claim, not a result.

There is a lot I would improve. My cluster runs on a single node, so I could not test true multi-node resilience or zone failures. The validation set has only 100 examples, so a larger set and several training seeds would make the model comparison meaningful. The teacher's paraphrased slots cost about 5% of the data, so a repair pass would help. The load tests ran on a shared two-vCPU runner, and I did not measure the saturation throughput of one replica. I would also run all the measurements again from the final repository, so that every number in the report comes from that repository and not from the rehearsal run.

My main observation from the whole project is that the model itself was the easy part. The hard, valuable work was everything around it: reproducible pipelines, honest measurement, automated gates and clean deployment. I now understand why industry talks about MLOps as a discipline of its own, and I feel far more confident taking a model from a notebook to a running, load-tested service.